CODE QUALITY AUDIT

TheOneWayGDA Platform

Comprehensive audit of the AI model comparison and statistical analysis platform, covering security, performance, correctness, and code quality across 50+ source files.

8 Critical vulnerabilities identified including insecure password hashing, bypassable XSS sanitization, and unauthenticated data exposure. 22 High-severity issues including cascade re-render patterns and memory leaks. Full remediation plan with effort estimates provided.

79
TOTAL ISSUES

50+
FILES AUDITED

17
PRIORITY FIXES

JUNE 15, 2026

Next.js 16 / React 19 / TypeScript / Prisma

Table of Contents

1. Executive Summary

This report presents a comprehensive code quality audit of TheOneWayGDA, an AI model comparison and statistical analysis platform built with Next.js 16, React 19, TypeScript, Prisma, and Zustand. The audit examined over 50 source files spanning workspace components, API routes, authentication, security utilities, hooks, and state management. The analysis was performed on June 15, 2026, against the main development branch.

The project demonstrates a substantial and ambitious feature set, including real-time AI model benchmarking, a full statistical analysis workspace with over 50 tests, AI-powered workflow automation, community features, and team collaboration. However, the audit uncovered a significant number of issues across security, performance, correctness, and code quality dimensions that require immediate attention before the platform can be considered production-ready.

Severity	Count	Key Themes
CRITICAL	8	SHA-256 password hashing, bypassable XSS sanitization, Rules of Hooks violation, unauthenticated data exposure, session forgery, unauthenticated arena manipulation, build failure (middleware + proxy conflict)
HIGH	22	Insecure CSRF fallback, token leak via URL params, cascade re-renders from Zustand, silent Excel import failure, localStorage data loss risk, missing body size limits, brute-force vulnerability, memory leaks in AiCopilot
MEDIUM	37	Pervasive any types, missing ARIA labels, blob URL leaks, unused dead code, statistical inaccuracy in t-test, fabricated cache metrics, i18n bundle bloat, stale closures in hooks
LOW	12	Unused imports, misleading comments, duplicated utilities, minor logic inconsistencies, edge cases in statistical functions

Table 1: Issue severity summary across all audited files

The most urgent finding is that the project currently fails to build due to a Next.js 16 configuration conflict between middleware.ts and proxy.ts. Beyond this build blocker, the highest-priority issues are the insecure password hashing implementation (using SHA-256 instead of bcrypt/Argon2), the bypassable XSS sanitization based on regex patterns, and a React Rules of Hooks violation in the EmailGate component that will cause runtime crashes. On the performance side, the entire Zustand store subscription pattern creates cascade re-renders that severely degrade the workspace experience, particularly with large datasets.

2. Build and Compilation Status

2.1 Build Failure (CRITICAL)

The project fails to build with Next.js 16 due to the coexistence of both **src/middleware.ts** and **src/proxy.ts**. Next.js 16 requires the proxy.ts convention exclusively and treats the presence of both files as a fatal error. The error message states: "Both middleware file and proxy file are detected. Please use proxy.ts only." This is a

complete build blocker that prevents any deployment.

The proxy.ts file was created as a migration from middleware.ts (as documented in its own comments), but the original middleware.ts was never removed. The fix is straightforward: delete src/middleware.ts and retain src/proxy.ts as the sole request interceptor. The proxy.ts already contains all the same WAF, rate limiting, and security header logic that was in middleware.ts, so no functionality is lost.

2.2 ESLint Errors (22 errors, 3 warnings)

The ESLint analysis revealed 22 errors and 3 warnings. The most critical of these is the 17 violations of the React Rules of Hooks in EmailGate.tsx, where useState, useRef, useEffect, and useCallback are called conditionally after an early return statement. This will cause "Invalid hook call" runtime errors in development and subtle bugs in production. Additionally, three dashboard pages (analytics, developers, settings) access useCallback-wrapped functions before their declarations inside useEffect, triggering the react-hooks/immutability rule. Two pages use synchronous setState inside useEffect, which causes cascading re-renders.

2.3 TypeScript Errors

Four TypeScript errors were found, all stemming from a missing vitest type declaration. The test files in src/lib/__tests__/ import from vitest but the @types/vitest package is not installed. These errors are non-blocking for production builds (test files are typically excluded) but indicate the test infrastructure is not fully configured.

3. Critical Security Issues

3.1 Insecure Password Hashing (CRITICAL)

The authentication module in **src/lib/auth.ts** (lines 4-9) uses SHA-256 with simple salt concatenation for password hashing. Despite the code comment claiming "scrypt", the actual implementation uses crypto.createHash("sha256") with password + salt as input. SHA-256 is a fast cryptographic hash designed for message integrity, not password storage. It can be brute-forced at billions of hashes per second on modern GPUs, making it trivially crackable for any password under 12 characters. Proper password hashing requires a memory-hard key derivation function such as bcrypt, scrypt, or Argon2, which are specifically designed to be computationally expensive and resistant to GPU-based attacks.

Recommendation: Replace the SHA-256 implementation with Node.js built-in crypto.scrypt() or the bcrypt npm package. This is the single highest-priority security fix in the entire codebase.

3.2 Bypassable XSS Sanitization (CRITICAL)

The sanitizeInput() function in **src/lib/security.ts** (lines 85-117) attempts to prevent cross-site scripting through a chain of regex replacements targeting HTML tags, JavaScript URLs, and event handlers. However, regex-based HTML sanitization is a well-documented anti-pattern that has been successfully bypassed in every major implementation that has attempted it, including WordPress, OWASP ESAPI, and others. A single-pass

regex approach can be defeated with simple nesting techniques such as inserting a fake tag that, when stripped, reassembles a valid malicious tag. The function also lacks handling for SVG-based XSS, CSS expression injection, and mutation XSS (mXSS) vectors.

Recommendation: Replace the regex-based sanitizer with a battle-tested library such as DOMPurify for client-side sanitization, or better yet, ensure that user input is never rendered as raw HTML. React's built-in JSX escaping already provides protection when rendering text content, so the sanitizer may be entirely unnecessary if raw HTML rendering is eliminated.

3.3 Auth Token Leak via URL Query Parameter (HIGH)

The getTokenFromRequest() function in **src/lib/auth.ts** (lines 38-41) accepts authentication tokens from URL query parameters as a fallback. Tokens embedded in URLs are exposed in server access logs, proxy logs, browser history, HTTP referrer headers, analytics tools, and potentially in shared links. This creates multiple leak vectors that an attacker can exploit to hijack user sessions. The standard practice is to transmit session tokens exclusively via HTTP-only, secure cookies.

Recommendation: Remove the query parameter fallback entirely. If a browser-only GET flow requires token passing, use a short-lived one-time code that is exchanged server-side for a proper session cookie.

3.4 CSRF Protection Weaknesses (HIGH)

Two significant issues were found in the CSRF implementation in **src/lib/security.ts**. First, the hashToken() function (lines 207-214) falls back to a simple DJB2-style integer hash when crypto.subtle is unavailable. This hash is trivially reversible and collidable, providing effectively zero security for CSRF token validation. Second, the CSRF salt is hardcoded as a string literal in the source code (line 202). If the codebase is ever exposed (open source, client-side bundle leak, or insider threat), all CSRF tokens become forgeable since the salt is known.

Recommendation: Move the CSRF salt to an environment variable (process.env.CSRF_SALT). Remove the insecure DJB2 fallback entirely; if crypto.subtle is unavailable, refuse to generate CSRF tokens and log an error rather than generating insecure ones.

3.5 Unauthenticated Data Exposure (CRITICAL)

Multiple API endpoints expose sensitive data without any authentication check. The **/api/feedback** GET endpoint returns all submitted feedback entries including user email addresses, IP addresses, and timestamps to any anonymous requester. The **/api/arena** POST endpoint allows unauthenticated creation of arena battles with arbitrary model names, prompts, and responses, which directly pollutes the ELO leaderboard database. The **/api/ai/copilot** endpoint uses a client-controlled x-visitor-id header for identification and accepts client-supplied session IDs for database upserts, meaning any user can potentially overwrite another user's conversation history by supplying a different sessionId.

Recommendation: Add authentication to all data-mutating endpoints. The feedback GET endpoint should require admin authentication. The arena POST endpoint should require authentication and validate that model IDs correspond to real entries in the database. The copilot endpoint should generate session IDs server-side and

never trust client-supplied IDs for database operations.

3.6 Session Expiry Not Checked (CRITICAL)

The **/api/ai/agent/route.ts** (line 100) validates sessions by checking only that the token exists in the database, without verifying that the session has not expired. The database schema includes an expiresAt field on user sessions, but the query does not filter on it. This means that any previously issued session token, even one that expired months ago, grants full access to the AI agent endpoint. This is particularly concerning because AI endpoints are the most expensive to operate and consume real API credits.

3.7 Weak Password Policy (HIGH)

The registration endpoint in **/api/auth/register/route.ts** (line 14) enforces only a 6-character minimum length for passwords. There are no requirements for uppercase letters, lowercase letters, digits, or special characters. Given that the password hashing is already weak (SHA-256), this compounds the vulnerability by allowing trivially guessable passwords.

4. Performance Issues

4.1 Cascade Re-renders from Zustand Store (CRITICAL)

The most impactful performance issue in the entire application is the Zustand store subscription pattern in **src/hooks/useWorkspaceHandlers.ts** (line 94). The hook calls useAppStore() without any selector, which subscribes to the entire store object. Every state change, whether it is a data cell edit, a new chat message, an output addition, or a variable selection, triggers a full re-evaluation of the hook. This recreates all 30+ useCallback functions and causes every consuming component to re-render.

The downstream impact is severe. The workspace page (page.tsx, line 41-114) uses useMemo(() => [...panels], [t, h]) where h is the entire return value of useWorkspaceHandlers(). Since h is a new object on every render (because the hook recreates all callbacks), the useMemo is completely defeated. All 8 panel JSX trees are reconstructed on every keystroke in the data editor. For large datasets, the DataEditorPanel renders up to 100 rows of input elements, each of which calls setData() replacing the entire data object on every keystroke, causing a cascade that copies potentially hundreds of thousands of values per character typed.

Recommendation: Use individual Zustand selectors (useAppStore(s => s.data), useAppStore(s => s.variables)) or the useShallow hook for grouped selections. Use useAppStore.getState() inside callbacks instead of closing over the store object. For the DataEditorPanel, implement a setCellValue method that performs targeted updates instead of replacing the entire data object. Consider React.memo on row components and virtual scrolling for large datasets.

4.2 i18n Bundle Bloat (HIGH)

The **src/lib/i18n.tsx** file embeds translations for 8 locales inline in a single client-side file, adding approximately 222 KB of JSON strings to every client bundle that uses the useTranslation() hook. Since the I18nProvider wraps

the entire application in the root layout, this payload is loaded on every page, including pages that may never use any translations.

Recommendation: Split translations by locale into separate JSON files and load them dynamically using import() or Next.js i18n routing. This would reduce the initial bundle by approximately 195 KB (the weight of the 7 unused locales).

4.3 Memory Leaks (HIGH)

Multiple memory leaks were identified across the codebase. The **AiCopilot.tsx** component (line 391-411) creates a setInterval for streaming simulation but never stores or clears the interval handle, so it continues running after component unmount. The **use-toast.ts** hook sets TOAST_REMOVE_DELAY to 1,000,000 milliseconds (over 16 minutes), meaning dismissed toast DOM elements persist in memory for extended periods. Multiple setTimeout-based deferred computations in useWorkspaceHandlers.ts (validate, clean, z-score, normalize, log transforms) have no cleanup mechanism if the component unmounts before the timeout fires. The **cache.ts** module creates four MemoryCache instances at module level, each with an uncleared setInterval, and the **security.ts** module has an unrefereced setInterval for rate-limit cleanup.

4.4 localStorage Data Loss Risk (HIGH)

The Zustand store in **src/lib/store.ts** (lines 733-739) uses the persist middleware to save projects, including full data, variables, and outputs, to localStorage. The browser localStorage has a quota of approximately 5 MB. For datasets with 50,000 rows and 20 columns, the serialized state can easily exceed this limit. When the quota is exceeded, Zustand's persist middleware silently fails to write, meaning the user's work is not saved and will be lost on page reload. There is no error handling or user notification for this condition.

Recommendation: Add onRehydrateStorage error handling to detect quota failures. Consider persisting only project metadata (names, timestamps, variable definitions) to localStorage and storing the actual data in IndexedDB, which has a much larger quota.

5. React and Correctness Issues

5.1 Rules of Hooks Violation in EmailGate (CRITICAL)

The **src/components/EmailGate.tsx** component (lines 68-76) calls useTranslation() and usePathname() hooks, then conditionally returns null on line 74 before calling useState, useRef, useEffect, and useCallback on lines 76-211. When the pathname changes during the component's lifetime (which happens during SPA navigation), the hooks below the conditional return are sometimes called and sometimes skipped, violating React's fundamental Rules of Hooks. This causes "Invalid hook call" errors in development and can lead to corrupted hook state in production, resulting in unpredictable component behavior.

Recommendation: Move all hook declarations above the conditional return statement. Apply the visibility/gating logic inside the JSX return instead, using a wrapper div with display:none or conditional rendering of child elements.

5.2 Error Boundary Infinite Loop (MEDIUM)

The **src/components/error-boundary.tsx** (lines 85-92) automatically resets all error state when the error count reaches 5. This causes the erroring child component to re-render, which will likely throw the same error again, incrementing the count back to 5. This creates an infinite render loop that can freeze the browser tab. The boundary should instead display a permanent fallback after reaching the error threshold.

5.3 useToast Listener Churn (HIGH)

The **src/hooks/use-toast.ts** (lines 177-185) uses useEffect with the entire state object as a dependency. Since state changes on every toast update, the event listener is removed and re-added on every render. This causes unnecessary work and creates a brief window during which updates can be missed. The dependency should be changed to an empty array since the setState callback reference is stable.

5.4 Stale Closure in useChatOptimized (HIGH)

The sendMessage callback in **src/hooks/useChatOptimized.ts** (line 163, 261) depends on the messages array in its closure. Due to debouncing and async operations, by the time the async attemptSend function executes, the messages value may be stale. This means the API receives an outdated conversation context, potentially leading to incoherent AI responses. A useRef-based pattern should be used to track the latest messages value.

5.5 Numeric Cell Cannot Be Cleared (HIGH)

In the DataEditorPanel (**WorkspacePanels.tsx** line 387), numeric cells use parseFloat(e.target.value) || 0. When a user clears a numeric field, parseFloat("") returns NaN, and NaN || 0 evaluates to 0. This means users can never clear a numeric cell to empty or null, which is a significant usability issue for data cleaning workflows where missing values need to be represented.

6. API Route Issues

6.1 Missing Request Body Size Limits (HIGH)

Most API routes lack request body size validation. Only /api/ai/route.ts and /api/ai/agent/route.ts have partial Content-Length header checks, and both are bypassable when the Content-Length header is omitted (chunked transfer encoding). Routes such as /api/validate, /api/clean, /api/feedback, and /api/arena accept arbitrarily large request bodies, creating a denial-of-service vector where an attacker can send multi-gigabyte payloads to exhaust server memory. The /api/clean route is particularly dangerous because it combines unbounded input with an O(n*m) Levenshtein distance computation that can consume significant CPU time.

Recommendation: Add framework-level body size limits in the Next.js route handler config and per-route Content-Length validation for all endpoints. Implement the Math.max(...array) to array.reduce() pattern to prevent stack overflow on large column counts.

6.2 SSRF Risk in Search Route (HIGH)

The **/api/search/route.ts** (lines 75, 97, 117) uses process.env.NEXT_PUBLIC_APP_URL for internal fetch calls. The NEXT_PUBLIC_ prefix means this value is exposed to the client-side JavaScript bundle, making it potentially manipulable. Additionally, the internal fetch calls do not include authentication headers, meaning any client that knows the internal URL can directly access the search endpoint with elevated privileges.

Recommendation: Replace NEXT_PUBLIC_APP_URL with a server-only environment variable such as INTERNAL_API_URL. Add authentication headers to all internal fetch calls.

6.3 Unbounded Database Queries (HIGH)

Several API routes execute unbounded database queries. The arena leaderboard computation in **/api/arena/route.ts** (lines 351-357) calls db.arenaBattle.findMany() with no take limit, fetching every battle ever created for ELO computation. This query grows without bound as users create more battles. The search parameter limit in the same file (line 310) has no upper bound, allowing a caller to request limit=999999 and fetch the entire table. The /api/feedback route uses file-based storage with a classic TOCTOU (time-of-check-time-of-use) race condition where concurrent POST requests can silently overwrite each other's data.

6.4 Internal Error Message Leakage (MEDIUM)

Both the login and register routes (**/api/auth/login/route.ts** line 67, **/api/auth/register/route.ts** line 79) return error.message directly to the client. Database errors, Prisma connection failures, and other internal error messages may contain sensitive information about the database schema, connection details, or internal infrastructure. The /api/health/route.ts (lines 28-33) exposes process.platform, process.version, and the endpoint map to unauthenticated callers.

6.5 Stripe Webhook Issues (MEDIUM)

The **/api/stripe/webhook/route.ts** has several issues: the getPlanFromPriceId function (line 30) defaults to "pro" for unrecognized price IDs, meaning an unknown or test price ID silently creates a pro subscription. Fragile type casts (line 104) bypass TypeScript checking. A redundant database query (lines 111-122) fetches the same subscription record twice. The webhook verification failure returns 401 instead of the Stripe-recommended 400 status code.

7. Type Safety and Code Quality

7.1 Pervasive any Type Usage (MEDIUM)

The Zustand store in **src/lib/store.ts** uses the any type extensively, which propagates throughout the application. The OutputItem.content field (line 19) is typed as any, preventing any type checking on output content. The core data structure (lines 79, 91) uses Record for the data field, meaning column values have no type constraint. The agentResults field (line 113) is typed as any[] with no defined structure. The ScanTable.rows field (line 39) uses any[][]. These pervasive any types cascade downstream, requiring unsafe casts in WorkspacePanels.tsx (lines 782, 813, 993, 1112), useWorkspaceHandlers.ts (lines 210, 840, 961), and smart-data-import.tsx (lines 64, 282,

412).

Recommendation: Define a CellData type as string | number | null and use it throughout. Create a discriminated union for OutputItem content (table, chart, text variants). Define proper interfaces for agent results and scan results. This single change would eliminate dozens of downstream any casts.

7.2 Dead Code and Unused Exports (MEDIUM)

Significant dead code was identified across the codebase. The entire **src/hooks/useChatOptimized.ts** file (approximately 300 lines) is exported but never imported anywhere. The **src/lib/security.ts** file exports sanitizeInput, sanitizeForQuery, isValidEmail, validateFileUpload, generateCSRFToken, hashToken, getSecurityHeaders, and validatePayloadSize, but none of these are imported by any other file. The project instead uses its own implementations in proxy.ts and middleware.ts. The isValidEmail function is independently reimplemented in EmailGate.tsx and visitor/route.ts. The useWorkspaceHandlers hook returns importDialogOpen, handleFileUpload, and scanDialogOpen, none of which are consumed by any component. The store defines loadProjectsFromStorage as a no-op that is never called.

7.3 Duplicated Code (MEDIUM)

Several significant code duplications exist. The calcStats function is defined locally in useWorkspaceHandlers.ts (lines 17-39) and duplicates functionality already available in stats.ts. The middleware.ts and proxy.ts files contain nearly identical WAF, rate limiting, and security header logic. The isValidEmail function appears in at least three separate locations with slightly different regex patterns. The rate limiting implementation exists in both middleware.ts/proxy.ts and security.ts with different approaches.

7.4 Statistical Inaccuracy (MEDIUM)

The t-test implementation in **src/lib/stats.ts** (line 353-354) uses twoTailedPFromZ(), which computes p-values using the standard normal distribution (Z-distribution) rather than the Student's t-distribution. For small samples (n less than 30), the Z-approximation underestimates p-values, increasing the Type I error rate (false positives). The code includes a comment acknowledging this limitation. Additionally, the skewness and kurtosis calculations divide by the standard deviation without guarding against stddev equals zero (which occurs when all values are identical), producing Infinity or NaN results.

8. Accessibility Issues

Multiple accessibility barriers were identified across the workspace and UI components. In **WorkspacePanels.tsx**, the agent goal input (line 55) has no associated label, aria-label, or aria-labelledby, making it invisible to screen readers. Variable selection checkboxes (lines 362-366) use adjacent span text that is not programmatically associated with the checkbox inputs. Data editor cell inputs (lines 381-383) have no accessible label, so screen readers announce "edit, blank" with no context about which variable or row the cell belongs to. The GDPR consent banner and email gate overlay lack role="dialog" and aria-modal attributes, meaning screen readers do not recognize them as modal dialogs and users can tab behind them. The AI copilot chat message container has no aria-live region, so new messages are not announced to screen reader users.

Recommendation: Add aria-label attributes to all form inputs and interactive elements. Use proper label associations for checkboxes. Add role="dialog", aria-modal="true", and aria-labelledby to modal overlays. Add aria-live="polite" and role="log" to the chat message container. These are relatively low-effort fixes that significantly improve the experience for users with disabilities.

9. Prioritized Recommendations

Based on the severity and impact of all findings, the following prioritized action plan is recommended. Items are grouped into three phases: immediate (must fix before any deployment), short-term (should fix within the next sprint cycle), and medium-term (should be addressed in the following iteration).

Priority	Issue	File(s)	Effort	
P0	Delete src/middleware.ts to resolve build failure	middleware.ts	5 min	Build
P0	Replace SHA-256 with bcrypt/Argon2 password hashing	lib/auth.ts	1 hour	Security
P0	Fix Rules of Hooks violation (move hooks above return)	EmailGate.tsx	30 min	Correctness
P0	Add authentication to feedback GET and arena POST	api/feedback, api/arena	2 hours	Security

P0	Check session expiresAt in agent auth query	ai/agent/route.ts	15 min	Security
P1	Switch to individual Zustand selectors	useWorkspaceHandlers.ts	3 hours	Performance
P1	Implement setCellValue for targeted data updates	store.ts	2 hours	Performance
P1	Add request body size limits to all API routes	Multiple routes	2 hours	Security
P1	Fix memory leaks in AiCopilot, toasts, and timeouts	Multiple files	2 hours	Reliability
P1	Remove URL query param token acceptance	lib/auth.ts	15 min	Security
P1	Move CSRF salt to env, remove DJB2 fallback	lib/security.ts	30 min	Security
P2	Replace any types with proper interfaces	lib/store.ts	4 hours	Maintainability

P2	Split i18n by locale for smaller bundles	lib/i18n.tsx	3 hours	Performance
P2	Remove dead code, consolidate duplicates	Multiple files	3 hours	Maintainability
P2	Fix t-test to use t-distribution	lib/stats.ts	2 hours	Correctness
P2	Add ARIA labels and dialog roles for a11y	Workspace components	2 hours	Accessibility
P2	Migrate feedback from file to DB storage	api/feedback/route.ts	3 hours	Reliability

Table 2: Prioritized remediation plan with effort estimates